

```
import kagglehub

path = kagglehub.dataset_download("sulianova/cardiovascular-disease-dataset")

print("Path to dataset files:", path)
```

Using Colab cache for faster access to the 'cardiovascular-disease-dataset' dataset.
Path to dataset files: /kaggle/input/cardiovascular-disease-dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import shap
```

```
df = pd.read_csv("/root/.cache/kagglehub/datasets/sulianova/cardiovascular-disease-dataset/versions/1/cardio_train.csv", sep=';')
print("Shape of dataset:", df.shape)
df.head()
```

Shape of dataset: (70000, 13)

	id	age	gender	height	weight	ap_hi	ap_lo	cholesterol	gluc	smoke	alco	active	cardio
0	0	18393	2	168	62.0	110	80	1	1	0	0	1	0
1	1	20228	1	156	85.0	140	90	3	1	0	0	1	1
2	2	18857	1	165	64.0	130	70	3	1	0	0	0	1
3	3	17623	2	169	82.0	150	100	1	1	0	0	1	1
4	4	17474	1	156	56.0	100	60	1	1	0	0	0	0

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

```
df = df.drop(columns=['id'], errors='ignore')

print("Missing values:\n", df.isnull().sum())

X = df.drop(columns=['cardio'])
y = df['cardio']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

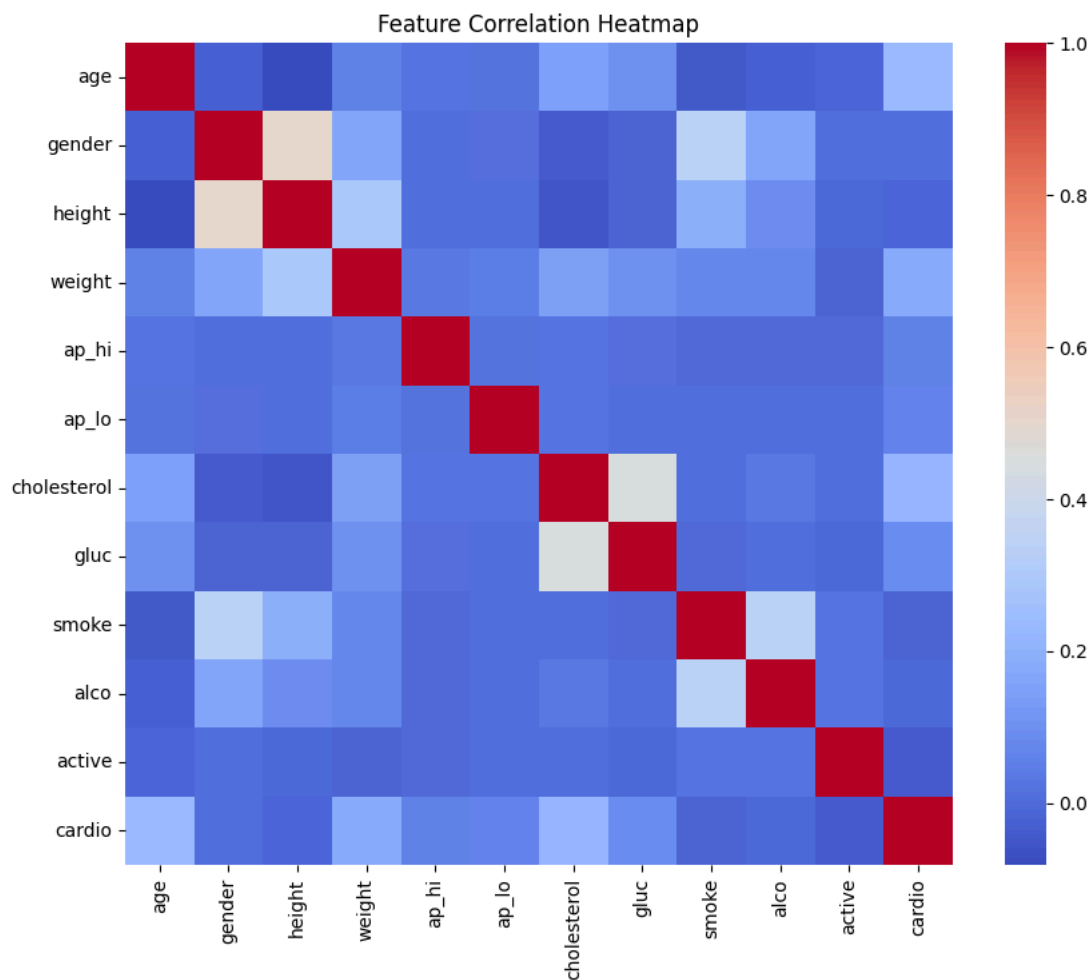
Missing values:

age	0
gender	0
height	0
weight	0
ap_hi	0
ap_lo	0
cholesterol	0
gluc	0
smoke	0
alco	0
active	0
cardio	0

dtype: int64

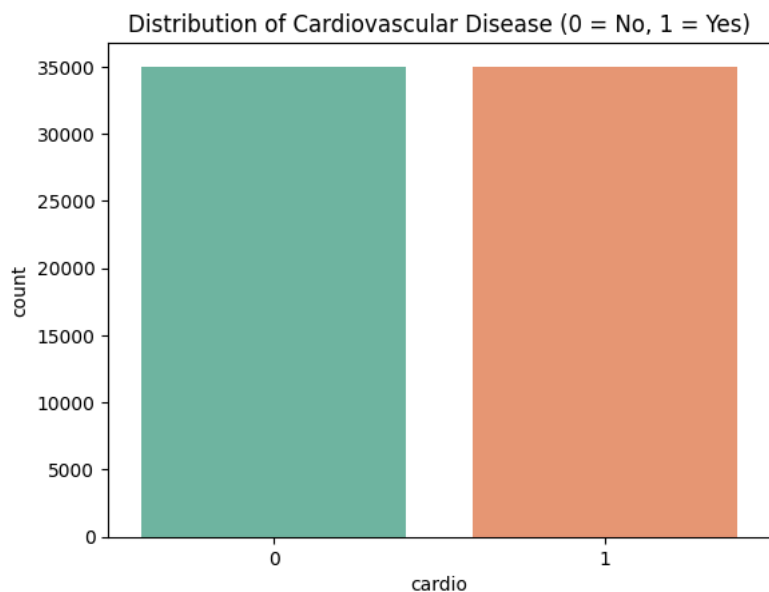
```
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), cmap="coolwarm", annot=False)
plt.title("Feature Correlation Heatmap")
plt.show()

sns.countplot(x='cardio', data=df, palette='Set2')
plt.title("Distribution of Cardiovascular Disease (0 = No, 1 = Yes)")
plt.show()
```



/tmp/ipython-input-3931672115.py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for
 sns.countplot(x='cardio', data=df, palette='Set2')



```
import xgboost as xgb
from sklearn.metrics import accuracy_score, classification_report

dtrain = xgb.DMatrix(X_train, label=y_train)
dtest = xgb.DMatrix(X_test, label=y_test)

params = {
    'tree_method': 'gpu_hist',
    'predictor': 'gpu_predictor',
```

```

    'objective': 'binary:logistic',
    'seed': 42,
    'verbosity': 1
}

num_round = 100
model = xgb.train(params, dtrain, num_round)

y_proba = model.predict(dtest)
y_pred = (y_proba >= 0.5).astype(int)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

```

/usr/local/lib/python3.12/dist-packages/xgboost/training.py:183: UserWarning: [14:09:55] WARNING: /workspace/src/common/error_msg.cc:27: The tree method

E.g. tree_method = "hist", device = "cuda"

```

bst.update(dtrain, iteration=i, fobj=obj)
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:183: UserWarning: [14:09:55] WARNING: /workspace/src/learner.cc:738:
Parameters: { "predictor" } are not used.

```

```

bst.update(dtrain, iteration=i, fobj=obj)
Accuracy: 0.735

```

```

Classification Report:
              precision    recall  f1-score   support

     0       0.72       0.78       0.75       6988
     1       0.76       0.69       0.72       7012

 accuracy          0.73       14000
 macro avg       0.74       0.74       0.73       14000
weighted avg       0.74       0.73       0.73       14000

```

/usr/local/lib/python3.12/dist-packages/xgboost/core.py:729: UserWarning: [14:09:55] WARNING: /workspace/src/common/error_msg.cc:27: The tree method

E.g. tree_method = "hist", device = "cuda"

```

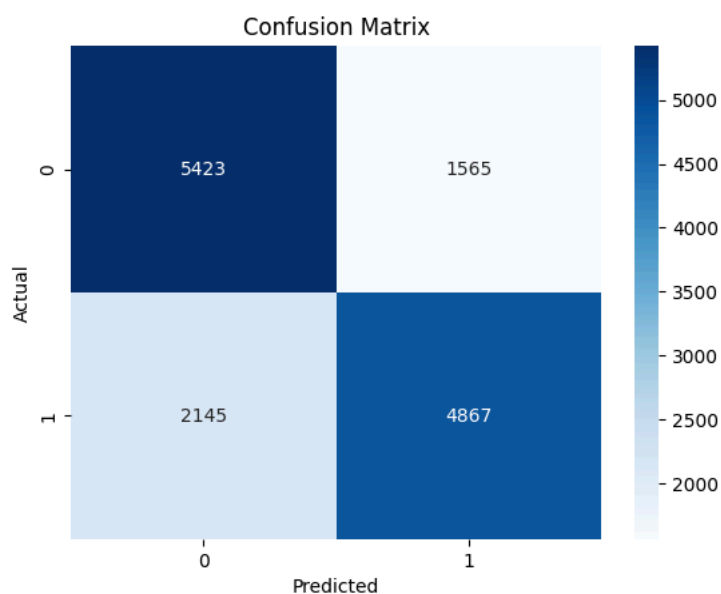
return func(**kwargs)

```

```

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

```



```

import numpy as np
import shap

```

```

explainer = shap.TreeExplainer(model)

```

```

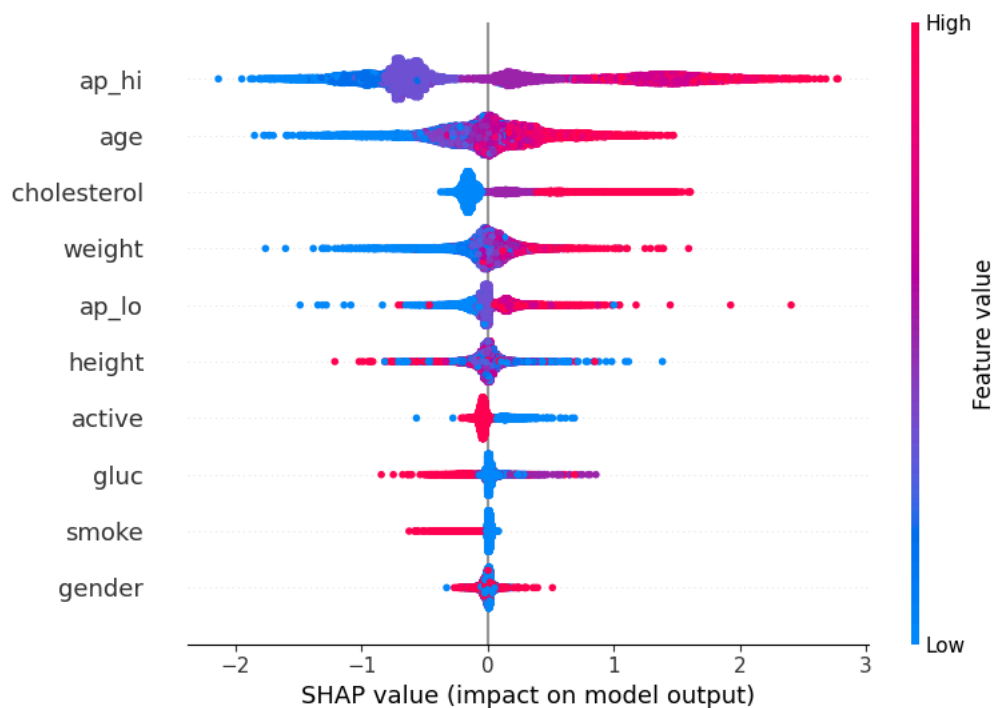
sample_size = 10000 if len(X_test) > 10000 else len(X_test)
sample_indices = np.random.choice(X_test.index, size=sample_size, replace=False)
X_test_sample = X_test.loc[sample_indices]

dtest_sample = xgb.DMatrix(X_test_sample)

shap_values_sample = explainer.shap_values(dtest_sample)

shap.summary_plot(shap_values_sample, X_test_sample, feature_names=X_test.columns, max_display=10)

```



```

import xgboost as xgb
import shap

X_test_single = X_test.iloc[[i]]

dtest_single = xgb.DMatrix(X_test_single)

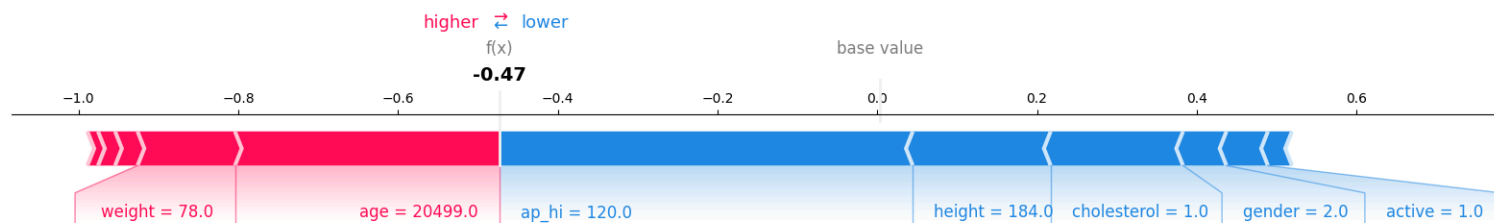
print("Predicted:", (model.predict(dtest_single) >= 0.5).astype(int))
print("True label:", y_test.iloc[i])

shap_values_single = explainer.shap_values(dtest_single)

shap.force_plot(explainer.expected_value, shap_values_single, X_test_single, matplotlib=True)

```

Predicted: [0]
True label: 0



1. What is Explainable AI, and why is it important in healthcare?

Explainable AI (XAI) refers to a collection of techniques and tools that make the working of AI models understandable to humans. In the healthcare domain, it is **extremely important** because AI models assist in making critical medical decisions that affect human lives. By using XAI, doctors can understand **why** an AI model made a certain prediction. It helps to:

- Build **trust** between AI systems and medical professionals
- Ensure **transparency** and **accountability**
- Support **ethical** and **safe clinical decisions**
- Comply with **regulatory standards** (like FDA and GDPR)

2. How does explainability improve trust between clinicians and AI systems?

Explainability allows clinicians to **see how and why** an AI model made a prediction. When doctors observe that the AI uses **relevant medical factors** (like age, blood pressure, cholesterol, etc.) to predict disease risk, they develop **confidence** in the system. This transparency makes AI a **collaborative partner** rather than a black-box system.

In summary:

- Builds **confidence** by providing clear reasons
- Helps doctors **verify** AI reasoning
- Encourages **human-AI collaboration** in decision making

3. How can clinicians validate the explanations provided by an AI system?

Clinicians can validate the AI explanations using the following methods:

1. **Compare** AI-derived feature importance with known **medical knowledge**.
2. Use **XAI tools** such as **SHAP** or **LIME** to visualize which features contributed most to a prediction.
3. Perform **case studies** or **pilot testing** on real patient data.
4. Check if AI consistently highlights **clinically relevant features** (like high BP, cholesterol, etc.).

This ensures that the model’s explanations are **trustworthy**, **accurate**, and **clinically meaningful**.

4. Differentiate between model-specific and model-agnostic XAI techniques.

Type	Description	Examples	
Model-Specific	These techniques are designed for a particular class of models and use the model’s internal structure for interpretation.	SHAP (for tree-based models), Grad-CAM (for CNNs)	Specific models
Model-Agnostic	These techniques can explain any machine learning model, regardless of its internal working.	LIME, Permutation Importance	Any black-box model

In this lab experiment:

We used **SHAP**, which is a **model-specific** technique optimized for tree-based models like **Random Forest**.

Conclusion

Explainable AI ensures that healthcare models are not just accurate, but also **transparent, interpretable, and trustworthy**. With XAI techniques such as **SHAP**, clinicians can confidently use AI predictions for **disease diagnosis, risk prediction, and treatment recommendations**.